Deployment-Aware Machine Learning for DoS Detection in Edge-Based IoT Networks

Samuel W. Almeida
State University of Ceará, Brazil
samuel.william@aluno.uece.br

Ivo A. Pimenta
State University of Ceará, Brazil
aguiar.pimenta@aluno.uece.br

Francisco V. Nobre
State University of Ceará, Brazil
valderlan.nobre@aluno.uece.br

Humberto P. Marques
State University of Ceará, Brazil
humberto.pires@uece.br

Waldir Moreira
Fraunhofer Portugal AICOS, Portugal
waldir.junior@aicos.fraunhofer.pt

Rafael L. Gomes
State University of Ceará, Brazil
rafa.lopes@uece.br

***Abstract*—The increasing adoption of Internet of Things (IoT) applications has expanded the attack surface of resource-constrained edge environments, creating the need for security mechanisms that combine accurate threat detection with efficient on-device execution. This work presents a deployment-aware Machine Learning (ML) solution for selecting DoS detection models for IoT networks, considering both detection quality and inference cost on edge devices. The proposed pipeline integrates feature-selection strategies, classification algorithms, hyperparameter optimization, and on-device high-load analysis. Additionally, a Pareto-based analysis demonstrates that maximizing classification performance alone does not necessarily lead to the most suitable model for operational deployment. The results highlight the importance of jointly considering detection quality and computational efficiency when deploying ML-based security mechanisms in resource-constrained IoT infrastructures, providing a practical basis for resource-aware security and intelligent edge computing.**

***Index Terms*—Machine Learning, IoT, Security, Edge.**

I. Introduction

Edge computing infrastructures represent a paradigm in which computation, storage, and intelligence are distributed across nodes closer to data sources, enabling low-latency and bandwidth-efficient services for applications such as the Internet of Things (IoT) [1]. However, edge environments are commonly characterized by limited computational resources, communication constraints, and latency-sensitive workloads [2]. Recent studies have also explored compact representations as a means of reducing the computational and memory requirements of learning tasks in resource-constrained environments [3]. Furthermore, the large volume of data generated at the edge increases pressure on network bandwidth and service latency, reinforcing the need for lightweight and computationally efficient detection mechanisms [4].

As a publish/subscribe-based Internet of Things application-layer communication protocol, MQTT comprises clients and brokers, enabling data decoupling and one-to-many communication through a simple and lightweight communication model. Its low bandwidth usage, low energy consumption, and low latency make it particularly suitable for resource-constrained IoT devices [5]. However, the limited resources and dynamic operating environments of these devices make traditional security mechanisms challenging to deploy and increase their exposure to network attacks. Among these threats, Denial-of-Service (DoS) attacks are particularly relevant, as large volumes of publishing, subscribing, or connection messages can exhaust node resources and disrupt normal services. Therefore, detecting malicious MQTT traffic while maintaining low computational overhead remains an important challenge for resource-constrained edge platforms.

Within this context, this work presents a Machine Learning (ML)-based approach for classifying MQTT traffic as either normal or under a DoS attack on an edge platform. The proposed approach compares the performance of multiple classifiers with different computational costs, evaluates different network feature-selection strategies, and measures model inference on a Raspberry Pi, aiming to identify a solution that provides a suitable balance between classification performance and time and resource constraints.

The main contribution of this work is a deployment-oriented methodology for selecting ML-based DoS detection models for resource-constrained IoT edge environments. Rather than selecting a detector solely according to classification performance, the proposed approach jointly considers feature selection, hyperparameter optimization, detection quality, and inference cost measured on a real edge device. To this end, multiple feature-selection strategies and classifiers are systematically evaluated, the resulting models are deployed on a Raspberry Pi under stress-test conditions, and a Pareto-based analysis is used to identify non-dominated solutions according to classification quality and computational efficiency. This methodology provides practical evidence of the trade-offs involved in deploying ML-based security mechanisms at the edge and supports model selection according to different operational requirements.

The remainder of this paper is organized as follows. Section II presents the related work. Section III presents the classification sequence, implementation architecture, and metrics used. Section IV presents the offline and Raspberry Pi results, as well as the Pareto analysis used to select the final model. Finally, Section V presents the conclusions of the study.

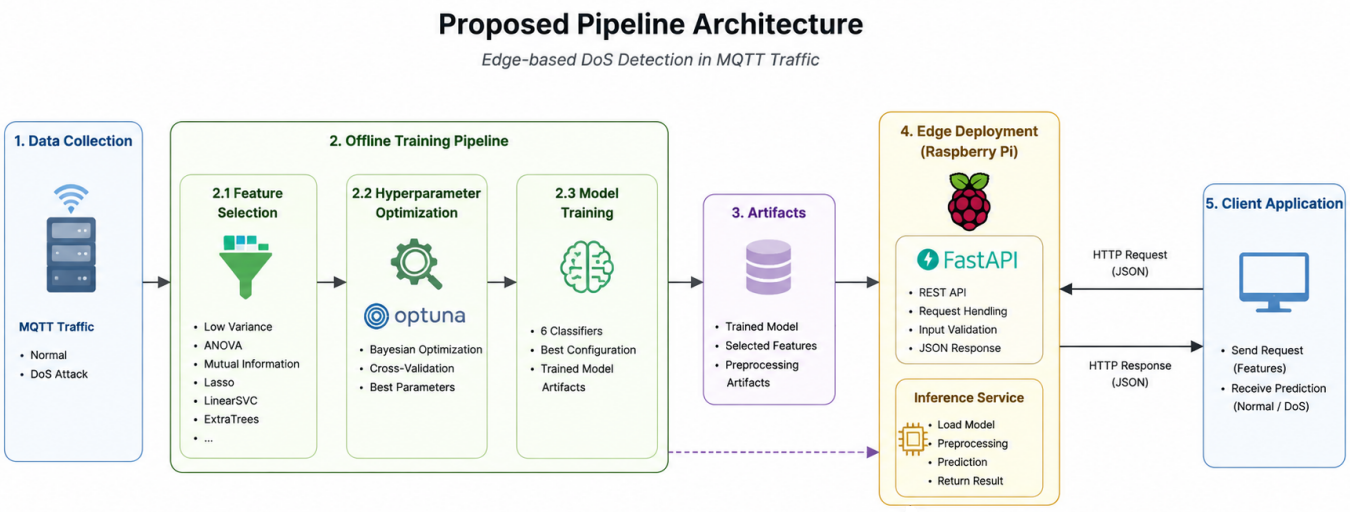


Fig. 1. Overview of the proposed pipeline and architecture for edge-based MQTT traffic classification.

II. Related Work

This section reviews recent work on the detection of attacks in MQTT-based IoT networks, with emphasis on the machine learning techniques employed and the constraints of resource-constrained devices.

Swain et al. [6] detect MQTT DoS attacks through communication-sequence anomalies, while Zeghida et al. [7] combine GAN-based augmentation with hybrid deep learning for MQTT intrusion detection. Saheed and Chukwuere [8] further explore explainable and privacy-preserving intrusion detection for industrial IoT environments.

Resource efficiency is also a major concern in constrained IoT devices. Tuyishime et al. [9] apply statistical feature selection to reduce dimensionality in MQTT traffic, whereas D'ıaz-Longueira et al. [10] show that shallow learning models can provide high detection performance with low computational requirements.

Overall, previous studies demonstrate the feasibility of ML-based MQTT attack detection and the benefits of lightweight models and feature reduction. However, these aspects are generally evaluated separately. In contrast, the proposed approach jointly considers feature selection, classification performance, and on-device inference cost, with deployment on a Raspberry Pi to assess the trade-off between detection quality and computational efficiency.

III. Proposal

This section presents the proposed Machine Learning-based pipeline and deployment architecture for DoS attack detection in IoT environments. Figure 1 provides an overview of the complete process, from data collection and preprocessing through feature selection, model training and hyperparameter optimization, offline evaluation, and deployment on a Raspberry Pi through the inference API. The architecture also includes the stress-testing and evaluation stages, which consider both classification performance and inference efficiency to identify suitable models for resource-constrained edge environments.

The implementation of the proposal (and the data MQTT Under Attack Dataset [11] in the experiments) is available in the repository of the authors[1]. The selected subset contains 94,625 records initially described by 66 predictor attributes and a binary label representing normal and DoS traffic. The normal class contains 49,111 records, while the DoS class contains 45,514 records, corresponding to 51.90% and 48.10% of the observations, respectively. This distribution does not present a substantial class imbalance.

The dataset is stratified into training and test partitions using an 80/20 split with random seed 42, resulting in 75,700 training and 18,925 test observations. The training partition is used for feature selection, hyperparameter optimization, and model fitting, whereas the test partition is reserved exclusively for final evaluation.

The preprocessing stage assumes that extreme values in network data may represent legitimate events, such as timeouts, slow connections, and traffic bursts. Therefore, no outlier removal or capping technique is applied, preserving these values for model training.

The initial dataset contains 67 columns, including the target label. During data cleaning, 46 columns considered unsuitable for deployment were removed, including absolute timestamps, IP and MAC addresses, TCP ports, frame identifiers, textual fields, payloads, and metadata generated by Wireshark. After this step, 20 predictor variables remained, in addition to the target label, as presented in Table I. Two additional temporal features, *publish_gap* and *connect_gap*, were then incorporated to represent the time intervals associated with MQTT

[1]The source code, experimental configurations, dataset preprocessing scripts, trained models, and artifacts required to reproduce the experiments are publicly available at https://github.com/William-SWS/mqtt_under_attack.

PUBLISH and CONNECT messages, resulting in 22 candidate attributes before feature selection.

TABLE I
WIRESHARK ATTRIBUTES RETAINED AFTER DATA CLEANING.

Attribute	Attribute
frame.time_delta	*frame.cap_len*
frame.len	*mqtt.clientid_len*
mqtt.conack.flags.reserved	*mqtt.conack.flags.sp*
mqtt.conack.val	*mqtt.conflag.cleansess*
mqtt.conflag.passwd	*mqtt.conflag.qos*
mqtt.conflag.reserved	*mqtt.conflag.retain*
mqtt.conflag.uname	*mqtt.conflag.willflag*
mqtt.kalive	*mqtt.len*
mqtt.msgtype	*mqtt.qos*
mqtt.retain	*mqtt.topic_len*

The excluded features include instance identifiers such as *frame.number* and *frame.md5_hash*, which may memorize individual observations without providing generalizable predictive information; absolute timestamps such as *frame.time.epoch*, whose capture time does not characterize the traffic type; textual payload-related fields such as *mqtt.username* and *mqtt.passwd*, which are session-specific and may introduce privacy concerns; and Wireshark-specific artifacts such as *frame.protocols* and other analysis metadata that are not necessarily available in production traffic.

A. Feature Selection

After preprocessing, each record is represented by 22 candidate attributes. Feature selection is fitted exclusively on the imputed training partition. Six feature-selection strategies are evaluated: LowVariance, ANOVA, Mutual Information, Lasso, LinearSVC, and ExtraTrees, representing approaches based on variance, statistical relevance, mutual information, regularization, and model-based feature importance.

- **LowVariance** [12]: Traditionally, variance measures the spread of samples from a dataset. Thus, a low variance value indicates proximity of the samples in the set.
- **ANOVA** [13]: It is a statistical technique used to assess differences between group means and determine if they are due to specific factors or occur by chance.
- **Mutual Information** [4]: It is used to identify features that share a large amount of information with the target variable, making it a powerful metric in problems where the relationship between variables may be nonlinear.
- **Lasso** [12]: It fits a linear model regularized with the L_1 norm and ranks attributes according to the absolute value of their coefficients.
- **LinearSVC** [14]: Linear Support Vector Classification (SVC) is a linear model that estimates sparse coefficients based on important characteristics, just like the Lasso model.
- **ExtraTrees** [15]: The Extremely Randomized Trees algorithm builds a set of non-pruned decision or regression trees completely at random. Unlike other tree-based methods, it divides the nodes by choosing cut points entirely randomly and uses the entire training set to grow the trees.

B. Hyperparameter Optimization

Each of the six feature representations is evaluated with six classification models: Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), Gaussian Naive Bayes (GNB), Decision Tree (DT), Random Forest (RF), and Gradient Boosting (GB). The combination of the six feature-selection strategies with the six classifiers results in 36 selector–classifier configurations. In addition, an optimized Support Vector Machine (SVM) is trained separately using the 12 attributes selected by LowVariance and is used as a reference model for comparison with the main configurations.

Hyperparameter selection is performed using Optuna with the Tree-structured Parzen Estimator (TPE) sampler [16]. Each optimization study performs 30 trials using five-fold stratified cross-validation and random seed 42.

Let M_j denote the jth classification model, $\boldsymbol{\lambda}_j$ a candidate hyperparameter configuration for that model, and Λ_j its corresponding hyperparameter search space. Let K denote the number of cross-validation folds and $F^{(k)}_{1,\mathrm{macro}}$ the macro F1-score obtained on validation fold k. For each candidate configuration, the macro F1-score is computed independently on each validation fold, and the optimization objective is defined as the mean of these values across the folds:

$$\boldsymbol{\lambda}_j^* = \arg\max_{\boldsymbol{\lambda}_j \in \Lambda_j} \frac{1}{K} \sum_{k=1}^{K} F^{(k)}_{1,\mathrm{macro}} \left(M_j(\boldsymbol{\lambda}_j) \right), \qquad K = 5. \quad (1)$$

As defined in Eq. (1), $\boldsymbol{\lambda}_j^*$ represents the hyperparameter configuration that maximizes the average macro F1-score over the five validation folds. Therefore, Optuna selects the configuration with the highest cross-validation mean macro F1-score for each classifier.

After each optimization study, the model with the best hyperparameter configuration is retrained using the complete training partition associated with the corresponding feature representation. For LDA and QDA, *StandardScaler* is applied within each cross-validation fold to prevent information from the validation partitions from being incorporated into the scaler fitting process.

C. Evaluation Metrics

Classification performance is evaluated using accuracy, precision, weighted F1-score, macro F1-score, and log loss. Accuracy measures the overall proportion of correctly classified observations, whereas precision quantifies the proportion of predicted positive instances that are correctly classified. And the F1-score represents the harmonic mean between precision and recall.

The precision is reported using support-weighted averages. The F1-score reported in the experimental results is also aggregated according to class support. Let C denote the set of classes, n_k the number of observations belonging to class

k, N the total number of observations, and $\mathrm{F1}_k$ the F1-score obtained for class k. The weighted F1-score is defined as

$$\mathrm{F1}_{\mathrm{weighted}} = \sum_{k \in C} \frac{n_k}{N} \mathrm{F1}_k. \tag{2}$$

As defined in Eq. (2), $\mathrm{F1}_{\mathrm{weighted}}$ assigns greater contribution to classes with more observations and is used as the F1-score reported in the classification results and in the classification-quality indicator adopted in the Pareto analysis.

In addition, macro F1-score is considered to evaluate performance by assigning equal importance to each class independently of its frequency. It is defined as

$$\mathrm{F1}_{\mathrm{macro}} = \frac{1}{|C|} \sum_{k \in C} \mathrm{F1}_k. \tag{3}$$

As defined in Eq. (3), $\mathrm{F1}_{\mathrm{macro}}$ assigns the same weight to each class, making it useful for assessing whether classification performance is maintained across both normal and DoS traffic. This metric is also used as the objective function during hyperparameter optimization, as described in Section III-B.

Finally, log loss is used to evaluate the quality of the predicted class probabilities by penalizing incorrect predictions made with high confidence. Lower log-loss values indicate probability estimates that are more consistent with the observed labels.

D. Deployment on the Edge Device

The selected models are serialized in *joblib*[2] format and deployed through the inference API. For each model, the time required to process the complete test partition of 18,925 records is recorded.

The per-record latency is obtained by normalizing the median batch processing time:

$$T_{\mathrm{record}} = \frac{T_{\mathrm{batch}}}{N_{\mathrm{batch}}}, \qquad N_{\mathrm{batch}} = 18{,}925. \tag{4}$$

As defined in Eq. (4), the normalized per-record latency is obtained by dividing the median batch inference time by the number of records processed in each batch. This value provides a common scale for comparing models but does not represent the processing time of independent single-record requests. The median is adopted as the primary measure because it is less sensitive to latency spikes caused by concurrency and process scheduling on the Raspberry Pi.

The endpoints (containerized API in FastAPI[3] on a Raspberry Pi) are implemented as asynchronous functions, while CPU-bound operations are executed in a thread through *asyncio.to_thread*. Four vicorn[4] processes handle incoming requests. The system configuration contains four logical CPUs and 3,795 MB of total memory. The selected execution configuration, denoted as *t16*, is summarized in Table II.

[2]https://joblib.readthedocs.io/

[3]https://fastapi.tiangolo.com/

[4]https://uvicorn.dev/

TABLE II
STRESS-TEST CONFIGURATION T16.

Parameter	Value
Endpoint	`POST /benchmark`
HTTP requests	50
Concurrency	15
Request timeout	300 s
Uvicorn workers	4
Records per batch and model	18,925
Timing observations per model	50
Total duration	738.791 s
Completed requests	50
Failures and timeouts	0

E. Stress Testing and Inference Analysis

Each of the 50 requests to the benchmark endpoint performs a complete inference for each model over 18,925 records. Consequently, each model classifies 946,250 records throughout the experiment. Considering the five LowVariance models included in the Pareto analysis, the experiment produces 4,731,250 classifications. ExtraTrees with Gradient Boosting using 15 attributes is also executed as a control configuration, but it is excluded from the Pareto analysis, which is restricted to the LowVariance representation.

The client limits the number of simultaneous requests to 15. HTTP latency reflects the complete benchmark request, whereas the reported model times correspond only to their respective prediction calls. As defined in Eq. (4), the normalized per-record latency is a batch-based ratio, not a measurement of independent single-record requests.

F. Model Comparison and Pareto Analysis

The final analysis considers the trade-off between classification quality and inference cost. The selected models are compared using their classification performance and median inference time normalized per record. To represent classification quality through a single indicator, accuracy and the support-weighted F1-score are combined with equal weights as

$$S = \frac{\mathrm{Accuracy} + \mathrm{F1}_{\mathrm{weighted}}}{2}. \tag{5}$$

As defined in Eq. (5), S represents the arithmetic mean of accuracy and the support-weighted F1-score, assigning equal importance to overall classification correctness and the balance between precision and recall. Higher values of S therefore indicate better classification performance according to these two criteria.

The Pareto analysis considers two conflicting objectives: minimizing the normalized per-record inference time T_{record} and maximizing the classification indicator S. A model is considered dominated if another model achieves a lower or equal T_{record} and a higher or equal S, with a strict improvement in at least one of the two objectives. Models for which no such alternative exists constitute the empirical Pareto frontier. This analysis allows the final model selection to consider both

detection performance and computational efficiency rather than classification quality alone. In this way, the proposed pipeline identifies models that provide suitable trade-offs for deployment on resource-constrained edge platforms.

IV. Results and Discussion

A. Classification Results

Among the evaluated feature-selection strategies, LowVariance was selected for the subsequent experiments because it provided the most favorable observed trade-off, reducing the representation from 22 to 12 attributes while achieving superior evaluation metrics and producing smaller trained models, which are better suited to resource-constrained environments.

Table III presents the metrics of the classifiers trained using the 12 LowVariance attributes, including the optimized SVM used as a reference. Gradient Boosting achieved the highest accuracy, precision, and F1-score, as well as the lowest *log loss*. Random Forest and Decision Tree obtained the next best results. The three simpler probabilistic classifiers achieved F1-scores close to 92.3%, while QDA and Gaussian Naive Bayes produced the highest probabilistic losses.

TABLE III
Metrics of the classifiers with the LowVariance attributes.

Model	Acc.	Prec.	F1	LL
LDA	0.9239	0.9326	0.9232	0.3764
QDA	0.9232	0.9317	0.9226	1.0225
GaussianNB	0.9234	0.9328	0.9227	1.2092
SVM	0.9291	0.9375	0.9285	0.2179
Decision Tree	0.9688	0.9699	0.9688	0.0891
Random Forest	0.9735	0.9742	0.9734	0.0848
Gradient Boosting	**0.9754**	**0.9757**	**0.9754**	**0.0714**

The selected Gradient Boosting model used 191 estimators, a maximum depth of 6, a learning rate of 0.035, and a minimum of 11 samples required for splitting. The Decision Tree used a maximum depth of 6, a minimum of 94 samples required for splitting, 27 samples per leaf, and *log2* feature selection. Although Gradient Boosting maximized the classification metrics, the difference in F1-score between the two models was only 0.66 percentage points, motivating a joint evaluation with inference cost.

Table IV compares the optimized SVM with the two models that define the practical trade-off between classification quality and computational cost. The SVM achieved relatively high weighted precision, but its accuracy and F1-score remained lower than those of the Decision Tree and Gradient Boosting models. Its *log loss* was approximately three times that of Gradient Boosting and 2.45 times that of Decision Tree.

TABLE IV
Comparison of classification metrics with the optimized SVM.

Model	Acc.	Prec.	F1	LL
Decision Tree	0.9688	0.9699	0.9688	0.0891
Gradient Boosting	0.9754	0.9757	0.9754	0.0714
Optimized SVM	0.9291	0.9375	0.9285	0.2179

B. Inference Time and Pareto Frontier

Table V presents the median inference times obtained in the t16 experiment. LDA processed the batch in the shortest time, followed by Decision Tree. Gradient Boosting required 692.885 ms, while Random Forest and SVM exhibited substantially higher costs. The SVM was approximately 1,526 times slower than Decision Tree when comparing the batch medians. To relate classification quality and computational cost, the classification indicator S, defined in Eq. (5), is used, together with the Pareto dominance criterion described in Section III-F.

TABLE V
Inference and Pareto classification results.

Model	S	Median (ms)	ms/record	Pareto
LDA	0.9236	82.234	0.00435	Yes
Decision Tree	0.9688	95.798	0.00506	Yes
Gradient Boosting	**0.9754**	692.885	0.03661	Yes
Random Forest	0.9735	4,897.255	0.25877	No
SVM	0.9288	146,159.933	7.72311	No

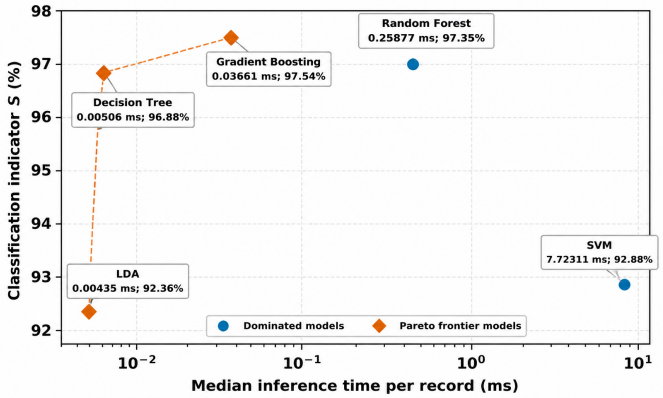


Fig. 2. Pareto frontier between the classification indicator and the median inference time normalized per record.

Figure 2 shows three non-dominated solutions. LDA defines the lowest-latency extreme but sacrifices classification quality. Gradient Boosting maximizes S; however, its median inference time is 7.23 times that of Decision Tree. Decision Tree occupies the intermediate position and represents the best observed operational trade-off: it increases S by 4.52 percentage points compared with LDA with an increase of only 13.564 ms per batch, whereas moving from Decision Tree to Gradient Boosting improves S by only 0.66 percentage points while adding 597.087 ms.

Random Forest was dominated by Gradient Boosting, which achieved higher classification quality with approximately seven times lower inference time. SVM was also dominated, combining lower classification quality with the highest latency. Thus, Gradient Boosting remains preferable when maximizing classification performance is the primary objective, whereas Decision Tree is the recommended choice for edge deployment when latency is a relevant constraint.

C. Final Discussion

The results obtained highlight the practical feasibility of deploying ML-based DoS detection mechanisms directly at the edge of MQTT-based IoT infrastructures. An important benefit of the proposed approach is that model selection is not based exclusively on classification performance. Instead, the methodology jointly considers detection quality and computational cost, allowing the identification of models that are better aligned with the limitations of resource-constrained devices. This aspect is particularly relevant in IoT environments, where gateways, embedded computers, and edge nodes must frequently execute security mechanisms while simultaneously supporting communication, data acquisition, and workloads.

The results also demonstrate that the model with the highest classification performance is not necessarily the most suitable choice for every deployment scenario. Gradient Boosting provides the best detection performance among the evaluated models and can therefore be attractive in environments where maximizing detection accuracy is the main requirement and additional computational resources are available. Conversely, the Decision Tree achieves classification performance close to that of Gradient Boosting while requiring substantially less inference time. Consequently, it represents a suitable alternative for latency-sensitive and computationally constrained edge environments. From a real-world perspective, the proposed architecture can be incorporated into IoT gateways, local MQTT brokers, or dedicated edge security nodes responsible for continuously analyzing network traffic before it reaches application or cloud services.

From this perspective, the contribution of this work focuses on investigating how security models should be selected when their actual deployment constraints are taken into account. The experimental results show that maximizing classification performance alone does not necessarily lead to the most suitable operational solution. This finding reinforces the importance of deployment-aware model selection, in which detection quality and computational efficiency are jointly considered according to the requirements and resource constraints of the target environment. Thus, the proposal provides a practical basis for selecting security models for edge-based IoT infrastructures rather than assuming that the most accurate classifier is necessarily the most appropriate one for deployment.

V. Conclusion

This work evaluated six classifiers combined with six feature-selection strategies for detecting DoS attacks in MQTT traffic. LowVariance reduced the representation from 22 to 12 attributes. Among the models evaluated with this representation, Gradient Boosting achieved the highest F1-score (0.9754), while Decision Tree maintained an F1-score of 0.9688 with substantially lower inference cost.

Future work should investigate more robust AI techniques, such as ensembles and environment structuring, which will be tested in the context of federated learning to verify that the accuracy improvements justify the additional inference cost on power-constrained devices over the top-performing classifiers studied here. Lastly, the environment will be formulated in a federated learning setting where models are trained among federated MQTT brokers without sharing any raw traffic so that the detectors can adapt to novel attack patterns as they emerge at every deployment location while maintaining data locality and minimizing network overhead.

Acknowledgments

The authors thank CNPq (No. 303877/2021-9 and No. 405940/2022-0) and CAPES (No. 88887.954253/2024-00) for the financial support.

References

[1] Y. Wang, C. Yang, S. Lan, L. Zhu, and Y. Zhang, "End-edge-cloud collaborative computing for deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 4, pp. 2647–2683, 2024.

[2] D. Tipper, P. Krishnamurthy, and A. Babay, "Availability analysis of multi-connectivity for providing urllc," *IEEE Networking Letters*, vol. 5, no. 4, pp. 223–226, 2023.

[3] E. S. Nascimento, I. A. Pimenta, M. H. Lee, T. de Araujo, and R. L. Gomes, "A decoupled embedding-based framework for efficient node classification on social networks," *Journal of Internet Services and Applications*, vol. 17, no. 1, pp. 214–223, 2026.

[4] I. A. Pimenta, M. H. Lee, L. F. Bittencourt, and R. L. Gomes, *IEEE Networking Letters*.

[5] F. Chen, Y. Huo, J. Zhu, and D. Fan, "A review on the study on mqtt security challenge," in *2020 IEEE International Conference on Smart Cloud (SmartCloud)*, 2020, pp. 128–133.

[6] M. Swain, N. Tripathi, and K. Sethi, "Identifying communication sequence anomalies to detect dos attacks against mqtt," *Computers & Security*, vol. 157, p. 104526, 2025.

[7] H. Zeghida, M. Boulaiche, R. Chikh, A. M. Bamhdi, A. L. B. Barros, D. Zeghida, and A. Patel, "Enhancing iot cyber attacks intrusion detection through gan-based data augmentation and hybrid deep learning models for mqtt network protocol cyber attacks," *Cluster Computing*, vol. 28, no. 1, p. 58, 2025.

[8] Y. K. Saheed and J. E. Chukwuere, "Cps-iiot-p2attention: Explainable privacy-preserving with scaled dot-product attention in cyber-physical system-industrial iot network," *IEEE Access*, vol. 13, pp. 81 118–81 142, 2025.

[9] E. Tuyishime, M. Martalò, P. A. Cotfas, V. Popescu, D. T. Cotfas, and A. Rekeraho, "Resource-efficient traffic classification using feature selection for message queuing telemetry transport-internet of things network-based security attacks," *Applied Sciences*, vol. 15, no. 8, 2025.

[10] A. Díaz-Longueira, J. Aveleira-Mata, Á. Michelena, A.-J. Piñón-Pazos, Ó. Fontenla-Romero, and J. L. Calvo-Rolle, "Shallow learning techniques for early detection and classification of cyberattacks over mqtt iot networks," *Sensors*, vol. 26, no. 2, 2026.

[11] J. Aveleira-Mata, H. Alaiz-Moretón, M. Bayón-Guitérrez, M. T. Gacía-Ordás, N. Prieto-Fernandez, and I. García-Rodríguez, "Mqtt_uad: Mqtt under attack dataset. a public dataset for the detection of attacks in iot networks using mqtt protocol," *Data in Brief*, p. 112167, 2025.

[12] W. L. Costa, M. M. Silveira, T. de Araujo, and R. L. Gomes, "Improving ddos detection in iot networks through analysis of network traffic characteristics," in *2020 IEEE Latin-American Conference on Communications (LATINCOM)*, 2020, pp. 1–6.

[13] I. Pimenta, D. Silva, E. Moura, M. Silveira, and R. L. Gomes, "Impact of data anonymization in machine learning models," in *Proceedings of the 13th Latin-American Symposium on Dependable and Secure Computing*, 2024, pp. 188–191.

[14] C.-C. Chang and C.-J. Lin, "Libsvm: A library for support vector machines," *ACM transactions on intelligent systems and technology (TIST)*, vol. 2, no. 3, pp. 1–27, 2011.

[15] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine learning*, vol. 63, no. 1, pp. 3–42, 2006.

[16] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019, pp. 2623–2631.